System Identification of a Nonlinear Two-Mass Spring-Damper System: Least Squares vs. Recurrent Neural Network

In this lab rotation, I worked with Prof. Caines on system identification of a nonlinear two-mass, two-spring, two-damper mechanical system, comparing classical least-squares (LS) against Recurrent Neural Networks (RNNs), motivated by the paper in [1]. The system has two carts connected in series to a fixed wall, each coupled to a nonlinear hardening spring with piecewise-linear function $\Gamma(d)$ along with a damper (see Appendix A for all governing equations). An external piecewise-constant force $u(t)$ drives Cart 1. The state vector is $\mathbf{x} = [x_1, \dot{x}_1, x_2, \dot{x}_2]^\top$ with $m = [\frac{1}{4}, \frac{1}{3}]$ kg, $c = [\frac{1}{4}, \frac{1}{3}]$ Ns/m, $k = [1, \frac{5}{6}]$ N/m (matching the reference paper). All code is at [3].

For the first phase, I started with data generation and Least Squares (LS) baselines. I simulated two spring variants (Fig. 1). The original system has a deadzone threshold, δ=1.0, and a shrunk variant with δ=0.25 (as suggested by Prof. Caines). Here, the spring enters a steeper linear region at much smaller displacements. For each variant, I generated a training trajectory (with seed 42, Fig. 2) and a held-out test trajectory (seed 99) using RK4 integration (Δt=0.01 s, T=200 s, 20 000 steps). I used both the raw states $\mathbf{x}$ and relative coordinates $\mathbf{z} = [x_1 - x_2, \dot{x}_1 - \dot{x}_2, x_2, \dot{x}_2]^\top$ to test my LS models. As a quantitative measure for model quality, I used NSE as defined in the paper: $\|\hat{y} - y\| / \|y\|$ (lower is better; NSE > 1 means divergence). I also implemented the Auto Regressive Moving Average (ARMA) for the LS approach. The ARMA predictor uses $\mathbf{x}_{k+1} = D\mathbf{x}_k + E_1 u_{k+1} + E_2 u_k$ to fit by pseudoinverse on all four combinations of data (Table 1, Appendix A). On the original raw springs, the LS model performs better in closed-loop rollout in either coordinate system (with test NSE$\approx$0.20 for raw data, and 0.31 for transformed z; Figs. 4 – 5) but it does not have perfect scores as a linear approach cannot model the deadzone's nonlinearity perfectly. We apply the same procedure to the shrunk springs, and it yields substantially lower NSE (test 0.08 raw, 0.15 transformed; Fig. 6 – 7): with δ=0.25 as the system spends most of its time in the linear region, making the overall linear approximation much more accurate than δ=1.0.

Next, I switched to the RNN. I initially built a vanilla open-loop Elman RNN ($h_k = \tanh(h_k W_{hh} + u_k W_{uh} + b_h)$, hidden size 32, 3,000 epochs), tracked the ground truth given true inputs at each step (Fig. 8) but it soon diverged immediately in autonomous rollout as the errors compound through the feedback path. So I redesigned the RNN's forward function to implement a closed-loop autoregressive model. This new model receives $[\mathbf{z}_k; u_k] \in \mathbb{R}^5$, predicts $\hat{\mathbf{z}}_{k+1}$, and that prediction feeds back as the next input. Working in transformed $\mathbf{z}$-coordinates (hidden 64, batch 16, Adam Learning Rate (lr) (lr=10^{-4}), gradient clipping), I compared four activations: **ReLU** (Fig. 9–10), **ELU** (Fig. 11), **tanh** (Fig. 12), and a physics-inspired deadzone variant (**NLT**) before the hidden update, z_1 is replaced by deadzone$(z_1, 1.0)$, zeroing the soft spring region to expose the underlying stiff dynamics more directly to the network (Fig. 13).

In the next phase, I experimented with two approaches: 2a and 2b. For 2a (Figs. 14–15) I used the best closed-loop ReLU configuration (h=64, z-coords) with a `ReduceLROnPlateau` scheduler (initial lr=0.01, factor 0.5, patience 50). Training loss reached 1.47×10^{-4} over 1,000 epochs, with the Learning Rate (LR) dropping to 3.9×10^{-5}. Despite this, the autoregressive NSE exceeded 1.7 on both sets. This shows that one-step teacher-forcing with a decaying LR over-optimizes for small loss but lacks stability. Approach 2b (Figs. 16–17) solved this using a physics-informed residual architecture. By feeding the 9-dim input $[\mathbf{z}_k; u_k; \hat{\mathbf{z}}^{\mathrm{LS}}_{k+1}]$, the RNN only learns the nonlinear residual: $\Delta\hat{\mathbf{z}}_{k+1} = \hat{\mathbf{z}}_{k+1} - \hat{\mathbf{z}}^{\mathrm{LS}}_{k+1}$. With the same scheduler, the model converged stably (test NSE: z_1=0.40, z_3=0.20), making this the best RNN result and matching the Z-coordinate LS baseline for z_3.

Ultimately, these experiments reveal that LS is a highly efficient baseline and that feeding back the LS prediction into the RNN (Exp. 2b) is the most effective RNN method tested. My future work can explore convex parameterization to improve the robustness of the vanilla RNN, like in [1], and extend the study to the full four-mass system.

References

[1] M. Revay, R. Wang, and I. R. Manchester, “A convex parameterization of robust recurrent neural networks,” *IEEE Control Systems Letters*, vol. 5, no. 4, pp. 1363–1368, Oct. 2021.

[2] M. Revay, R. Wang, and I. R. Manchester, "Recurrent equilibrium networks: Flexible dynamic models with guaranteed stability and robustness," IEEE Trans. Autom. Control, vol. 69, no. 5, pp. 2855-2870, 2023

[3] O. Gundra, “RNN Honours Rotation,” GitHub, 2026. [Online]. Available: `https://github.com/BraveKing15/RNN-Honours-Rotation`

Appendix A: Governing Equations and Results Summary

Nonlinear Spring Profile (δ = threshold):

$$\Gamma(d;\delta) = \begin{cases} d + 0.75\delta & d \le -\delta \\ 0.25\, d & |d| < \delta \\ d - 0.75\delta & d \ge \delta \end{cases} \quad \text{(original: } \delta{=}1.0,\ \text{shrunk: } \delta{=}0.25\text{)} \tag{1}$$

Equations of Motion (RK4, $\Delta t = 0.01$ s):

$$m_1\ddot{x}_1 = u - k_1\gamma(x_1) - c_1\dot{x}_1 + k_2\gamma(x_2 - x_1) + c_2(\dot{x}_2 - \dot{x}_1) \tag{2}$$
$$m_2\ddot{x}_2 = -k_2\gamma(x_2 - x_1) - c_2(\dot{x}_2 - \dot{x}_1) \tag{3}$$

LS ARMA Predictor (pseudoinverse):

$$\mathbf{x}_{k+1} = D\mathbf{x}_k + E_1 u_{k+1} + E_2 u_k = \Theta[\mathbf{x}_k; u_{k+1}; u_k], \qquad \Theta = X_{\text{next}} Z^\top (ZZ^\top)^{-1} \tag{4}$$

Closed-Loop Autoregressive RNN:

$$h_{k+1} = \sigma\big(h_k W_{hh} + [\mathbf{z}_k; u_k] W_{uh} + b_h\big), \quad \hat{\mathbf{z}}_{k+1} = h_{k+1} W_{hx} + b_x, \quad \mathbf{z}_{k+1} \leftarrow \hat{\mathbf{z}}_{k+1} \tag{5}$$

Physics-Informed Residual (Exp. 2b, input size 9):

$$\hat{\mathbf{z}}_{k+1} = \underbrace{D\mathbf{z}_k + E_1 u_{k+1} + E_2 u_k}_{\hat{\mathbf{z}}^{\text{LS}}_{k+1}} + \text{RNN}\big([\mathbf{z}_k; u_k; \hat{\mathbf{z}}^{\text{LS}}_{k+1}]\big) \tag{6}$$

Table 1: Normalized Simulation Error (NSE) Summary. Lower is better. NSE > 1 indicates divergence worse than predicting zero.

Method	Train NSE (avg)	Test NSE (avg)
Least Squares		
Original ± 1.0 — raw $\mathbf{x}$	0.1693	0.2001
Original ± 1.0 — transformed $\mathbf{z}$	0.2553	0.3089
Shrunk ± 0.25 — raw $\mathbf{x}$	0.0631	0.0795
Shrunk ± 0.25 — transformed $\mathbf{z}$	0.1181	0.1542
RNN (closed-loop, transformed $\mathbf{z}$, n_h=64)		
Exp. 2a: ReLU + LR Scheduler	> 1.7	> 1.7 (diverges)
Exp. 2b: LS Residual + LR Scheduler	≈ 0.27	≈ 0.30

Appendix B: Graphs and Data

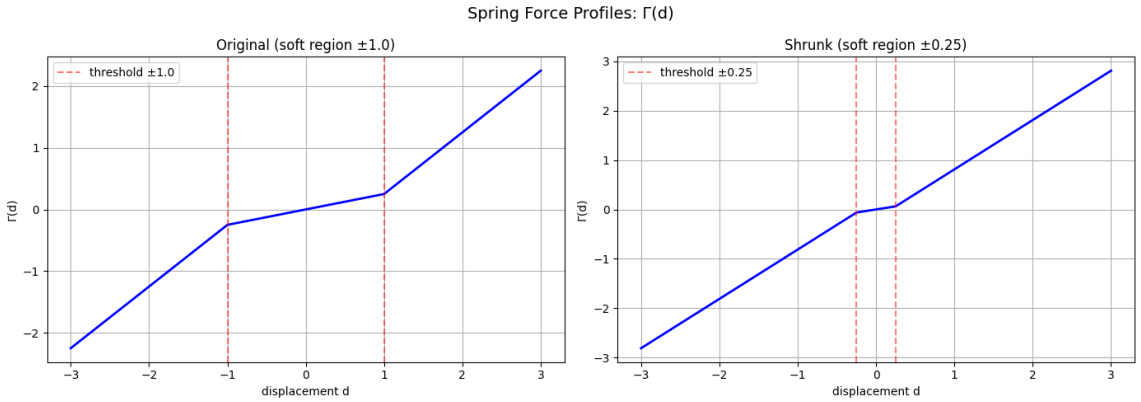

Figure 1: Spring Force Profiles. Left: original spring (δ=1.0) with a deadzone. Right: shrunk spring (δ=0.25), which enters the steeper linear region at much smaller displacements. The shrunk variant is more linear over typical amplitude ranges, making LS identification easier.

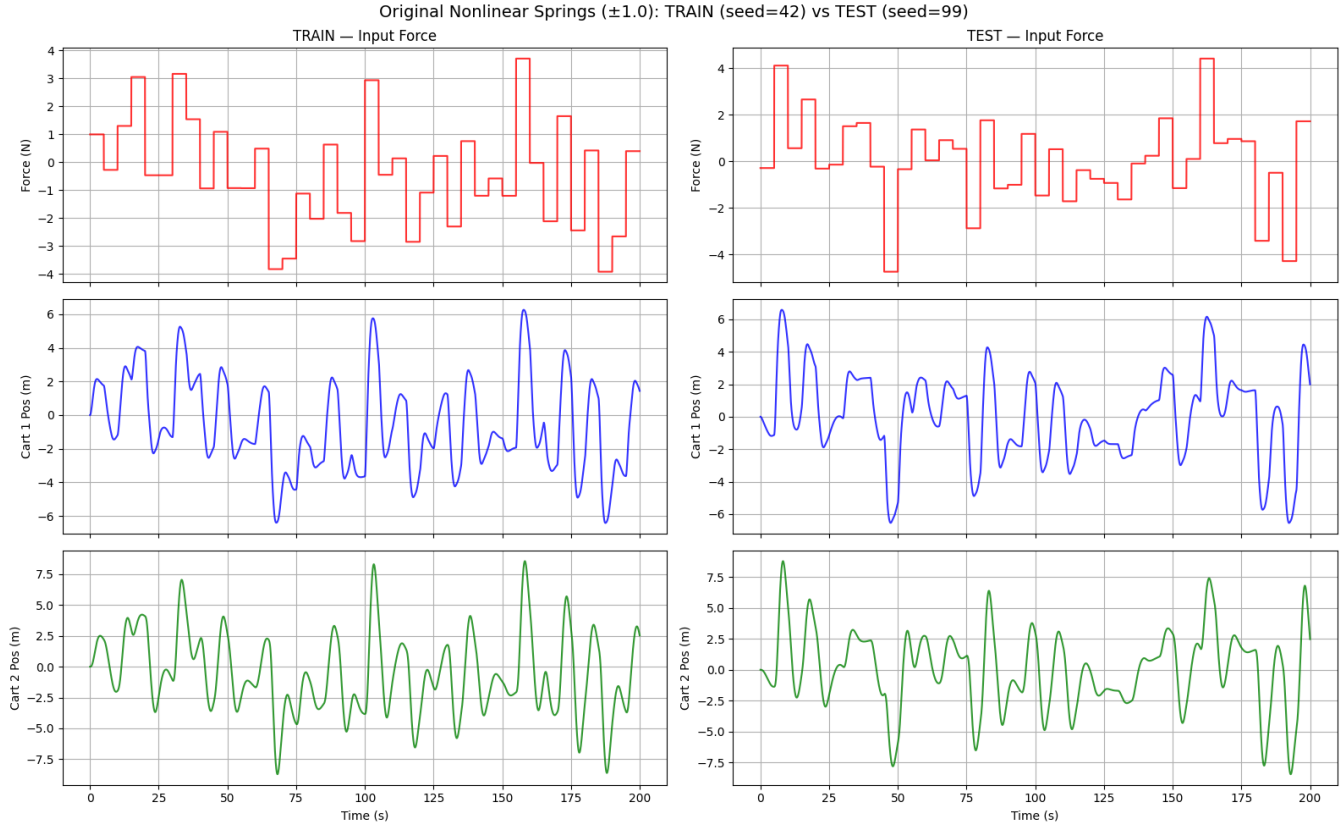

Figure 2: Training vs. Test Trajectories (Original Springs, δ=1.0). Left: training data (seed 42). Right: held-out test data (seed 99). Both use piecewise-constant input forces (switching every 5 s) over 200 s.

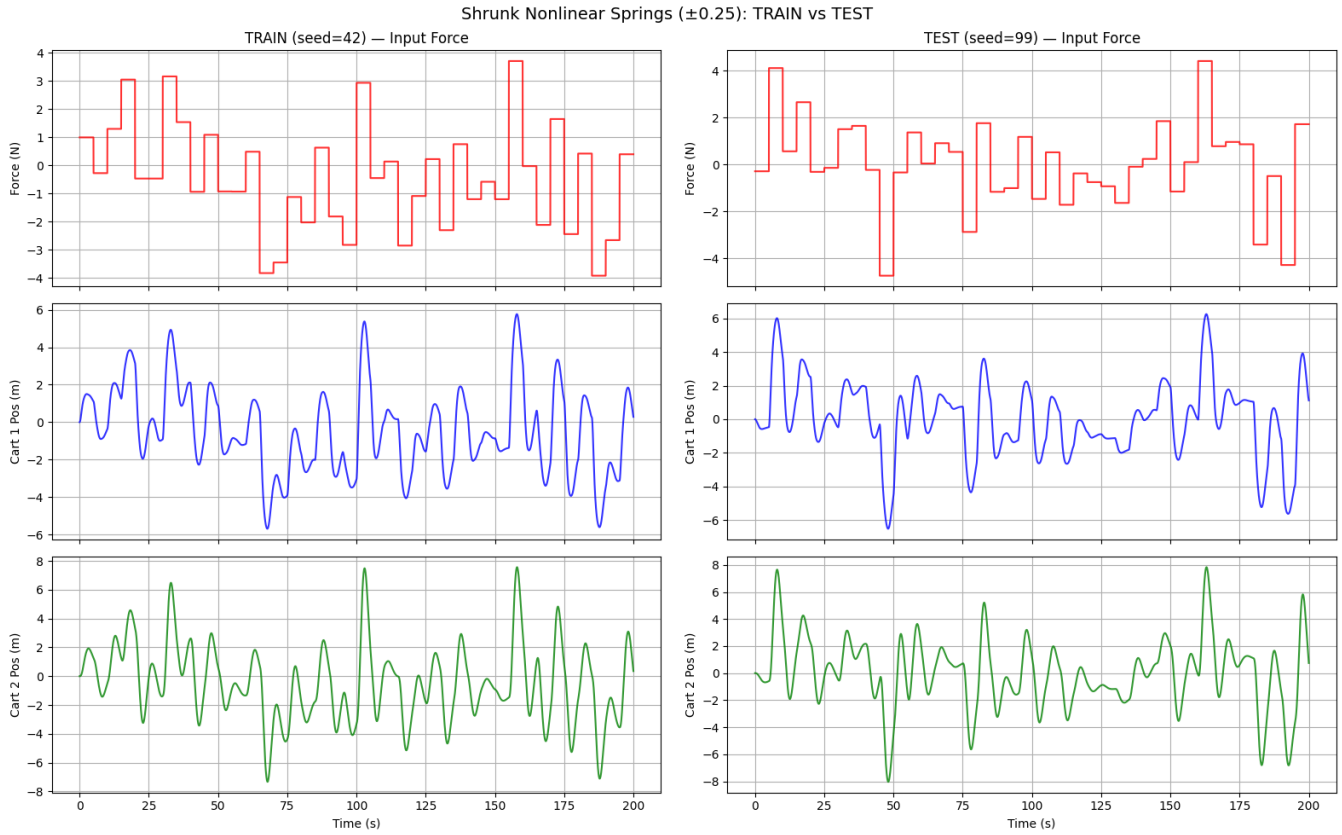

Figure 3: Training vs. Test Trajectories (Shrunk Springs, δ=0.25). Similar input. The reduced deadzone causes the system to settle faster with smaller oscillations compared to the original spring variant.

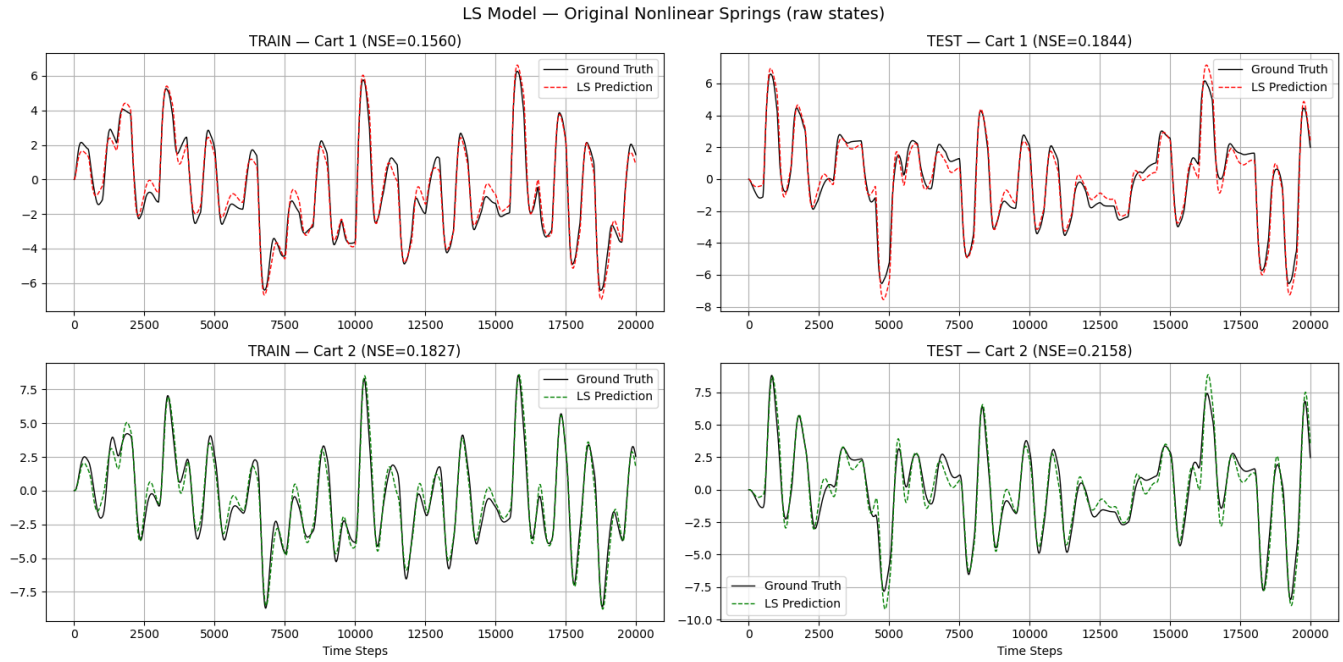

Figure 4: LS Model: Original Springs, Raw States. Train (left) and test (right) autoregressive rollout. NSE: Cart 1 train/test $\approx 0.15/0.18$, Cart 2 train/test $\approx 0.18/0.22$. The model approximately tracks the trajectory but diverges in the soft nonlinear region.

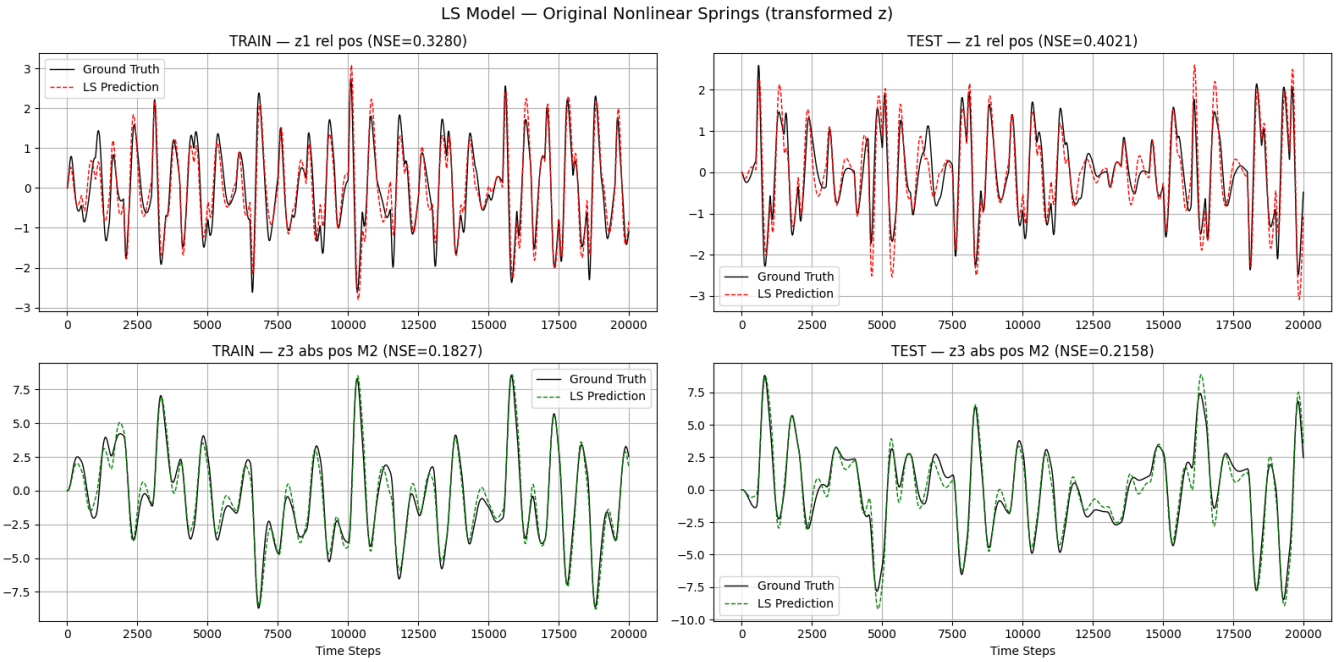

Figure 5: LS Model: Original Springs, Transformed **z**-Coordinates. Train/test NSE: $z_1 \approx 0.32/0.40$, $z_3 \approx 0.18/0.22$. The coordinate transformation does not improve the closed-loop fit. The nonlinearity is intrinsic and cannot be resolved by re-parameterization alone.

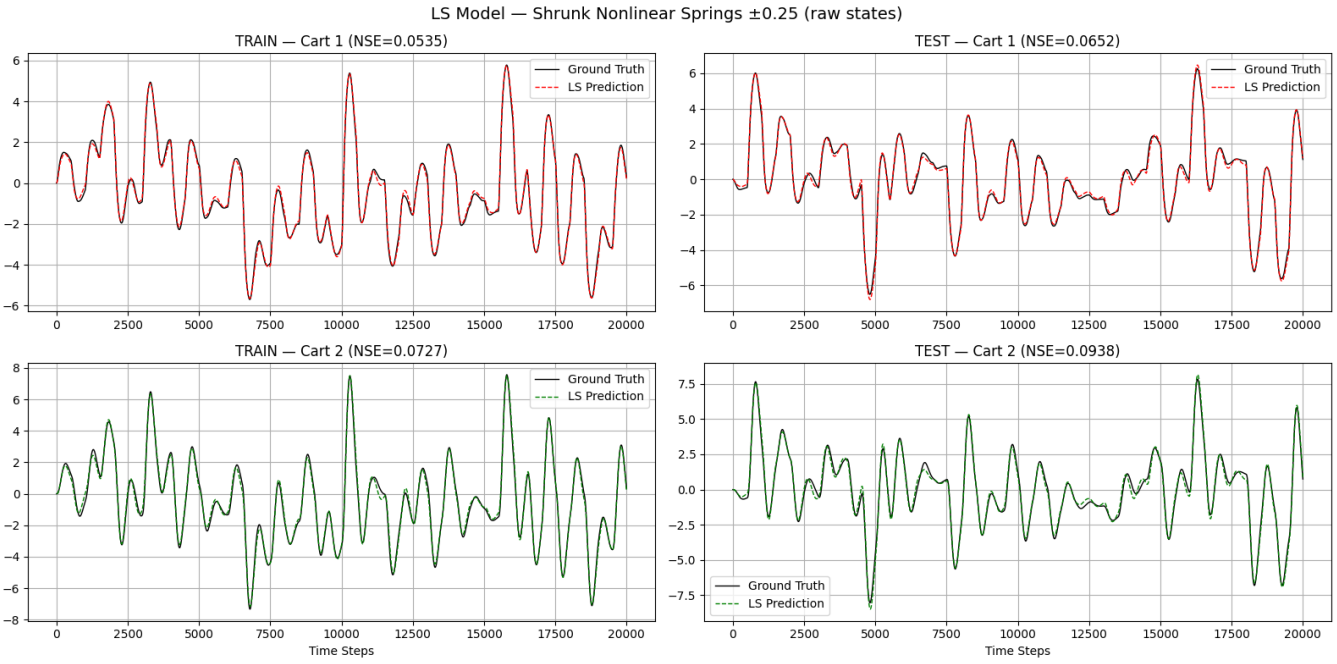

Figure 6: LS Model on Shrunk Springs (δ=0.25), Raw States. Train/test NSE: Cart 1 $\approx 0.05/0.07$, Cart 2 $\approx 0.07/0.09$. Substantially better than the original springs since the system frequently stays in the linear region, making the global linear model a better fit.

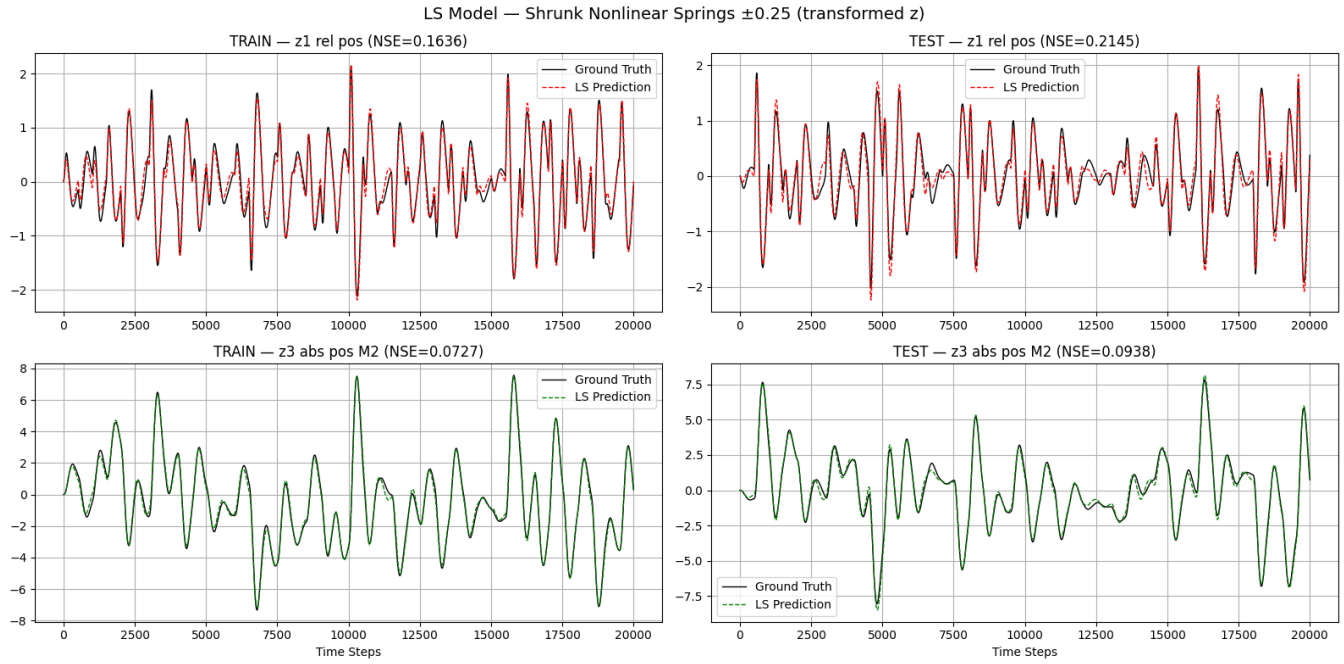

Figure 7: LS Model: Shrunk Springs (δ=0.25), Transformed $\mathbf{z}$. Train/test NSE: $z_1 \approx 0.16/0.21$, $z_3 \approx 0.07/0.09$. Almost no improvement over the original springs. Identical performance to the raw-state LS on the same data, confirming the transformation does not consistently help.

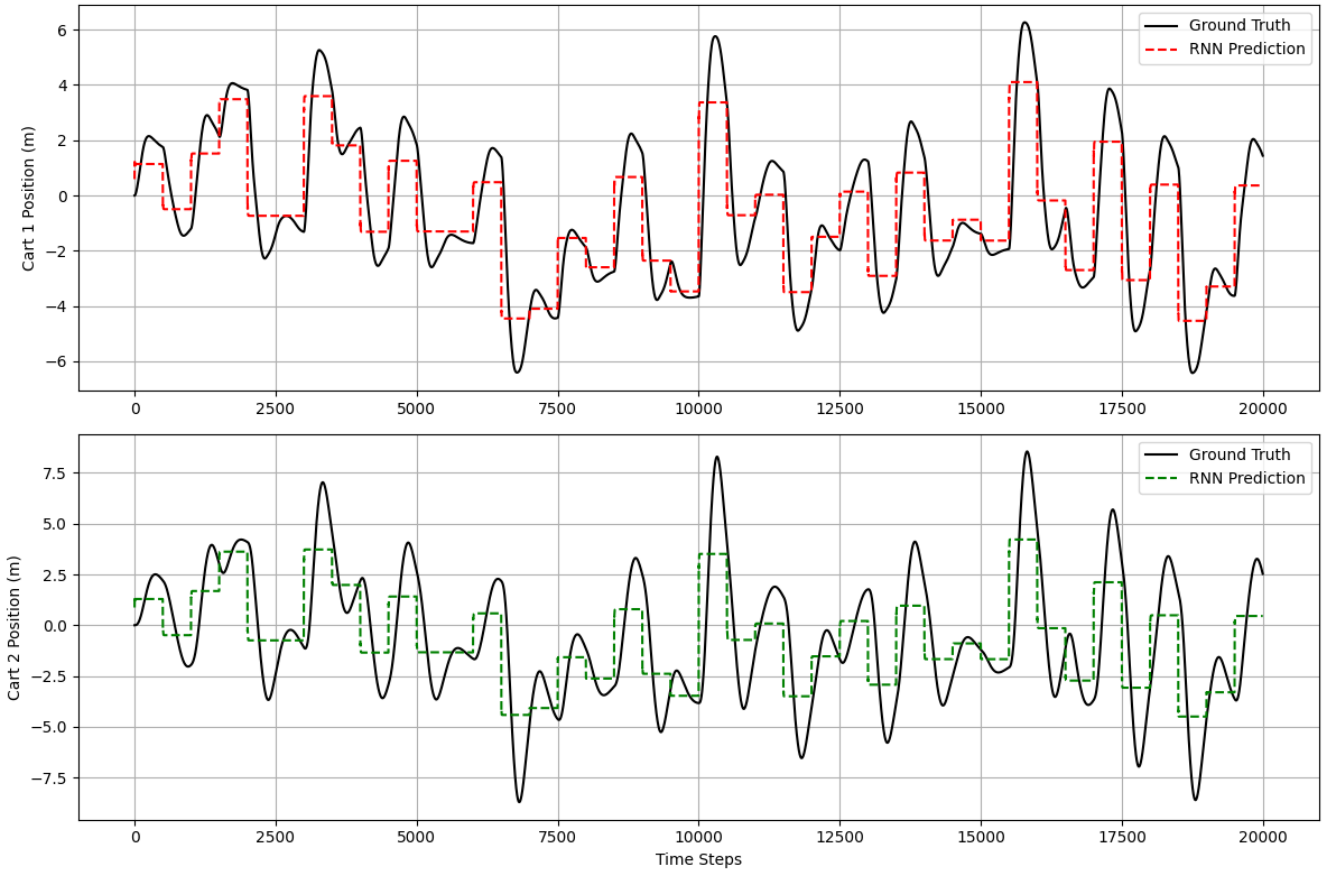

Figure 8: Open-Loop Vanilla RNN (hidden 32, tanh, raw states). Given the true input $u(t)$ at each step the model tracks the ground truth for both carts. However, this model diverges immediately when rolled out autoregressively, as prediction errors compound through the feedback path.

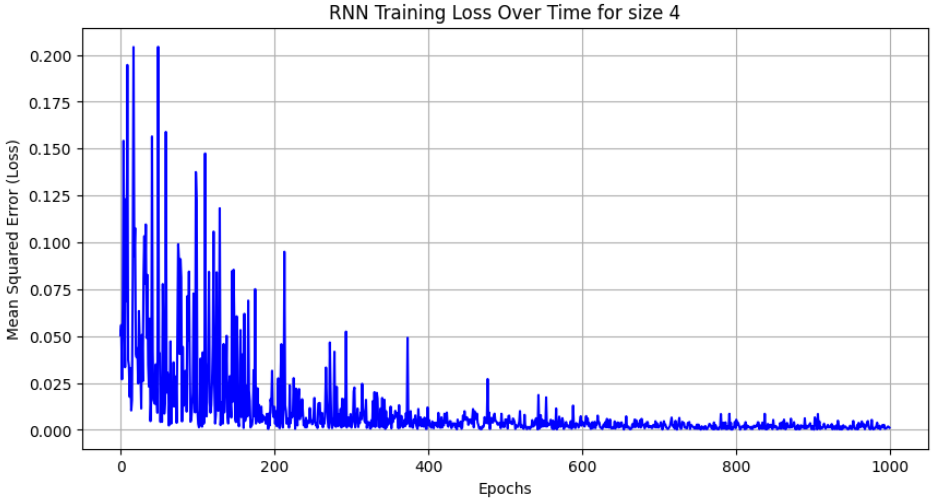

Figure 9: CL ReLU RNN Training Loss (hidden 64), **z**-coords, $n_{\text{epochs}} = 1000$). Loss decreases from ≈ 0.2 to ≈ 0.0015 over 1 000 epochs with Adam (lr=10^{-4}) and gradient clipping.

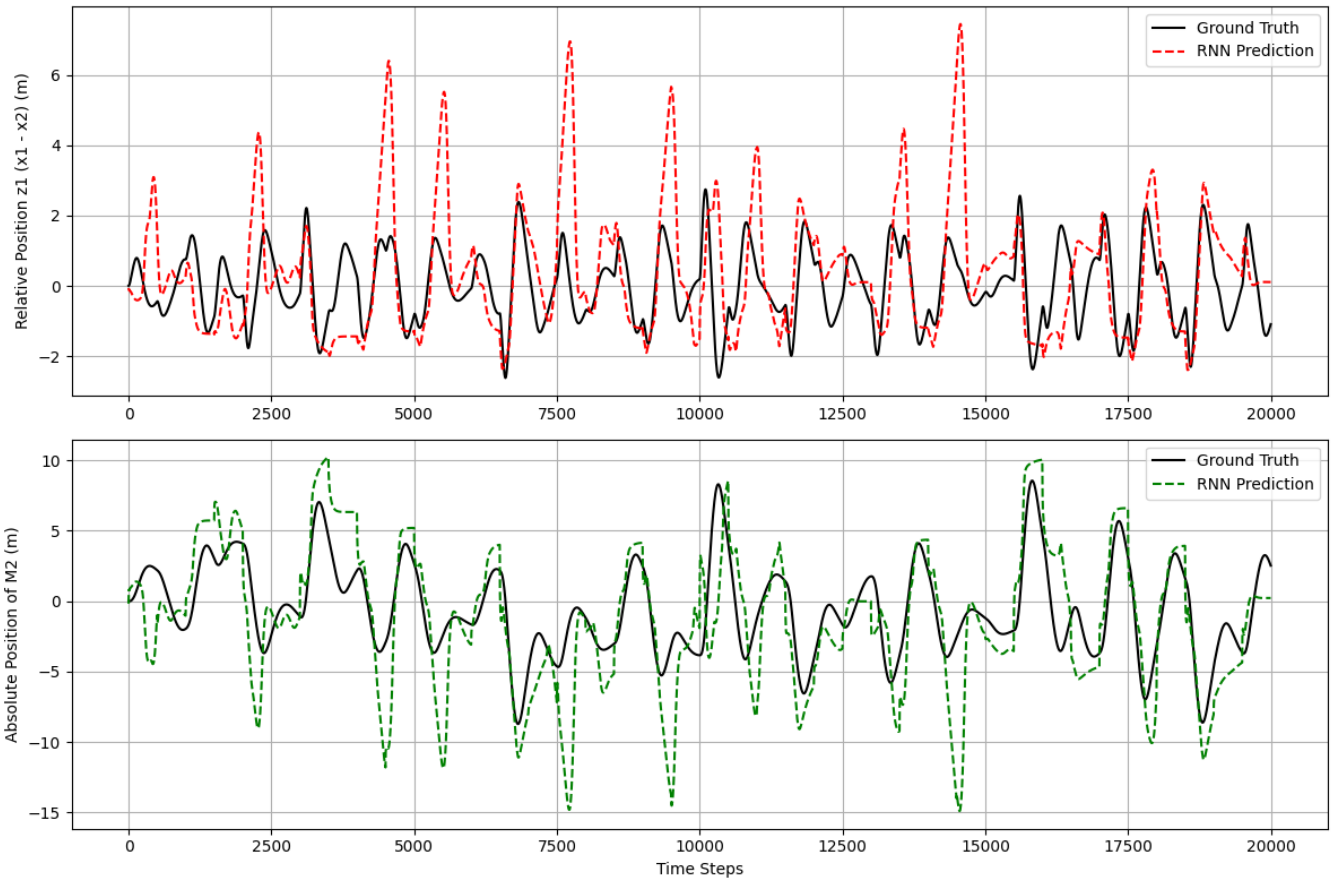

Figure 10: CL ReLU RNN Evaluation (hidden 64, **z**-coords). Train and test were both performed on the same dataset, yet the autoregressive rollout on z_1 (relative position) and z_3 (Cart 2 absolute position) still performs poorly on previously seen data.

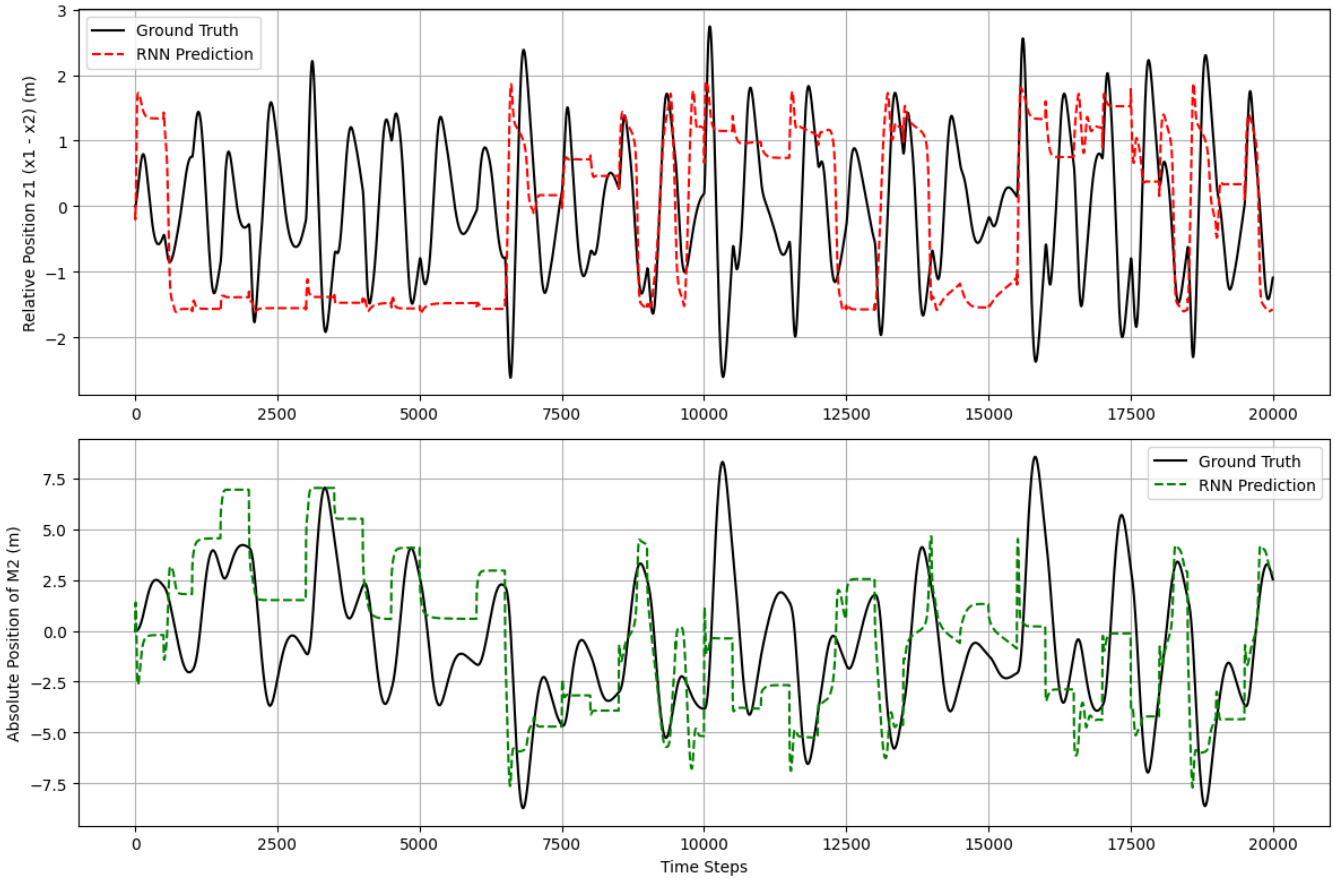

Figure 11: CL ELU RNN Evaluation (hidden 64, **z**-coords). Same architecture and training as Fig. 10 but with ELU activation. ELU's smooth negative saturation provides slightly different gradient flow but similar autoregressive tracking performance.

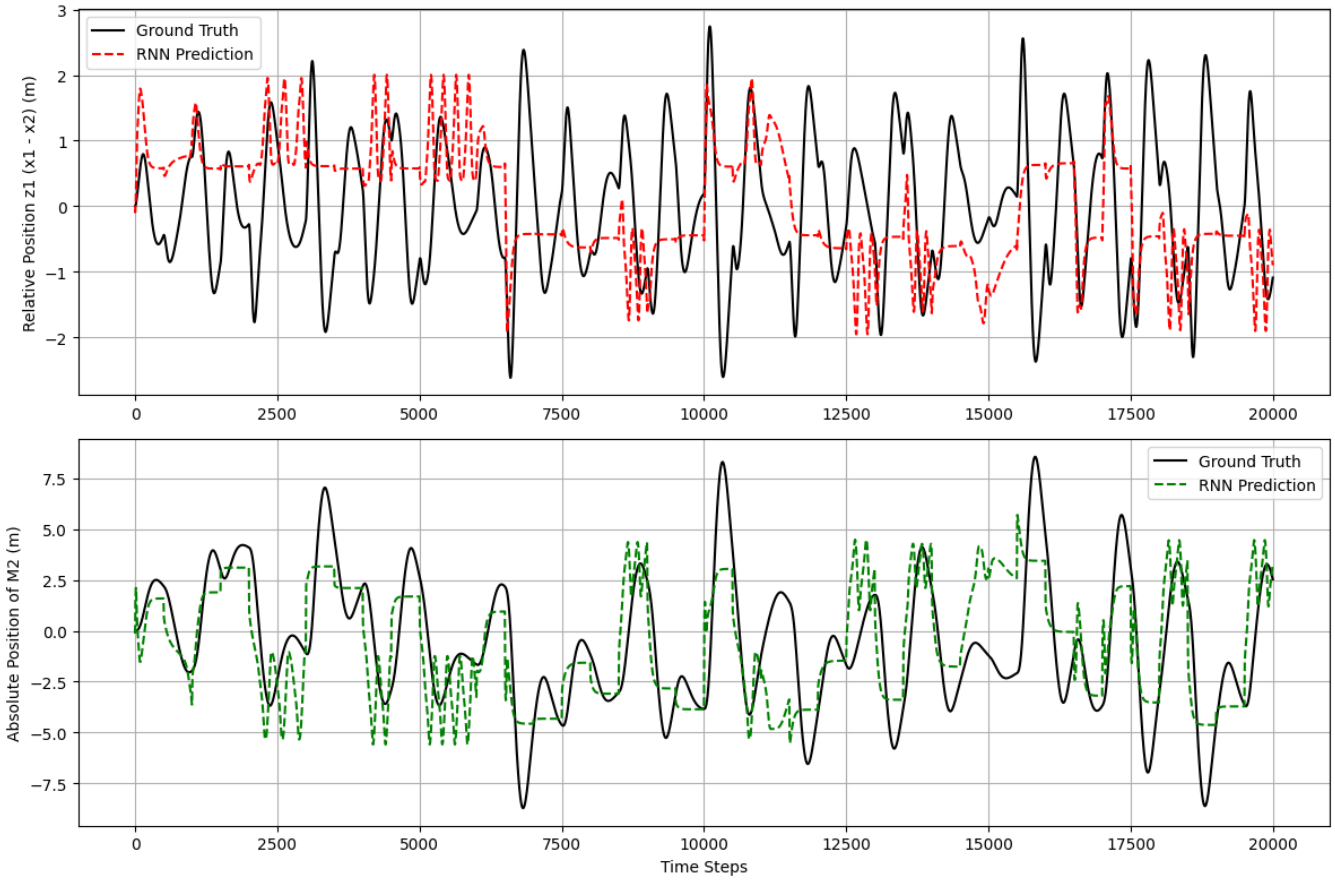

Figure 12: CL tanh RNN Evaluation (hidden 64, **z**-coords). tanh activation with the same closed-loop setup. The bounded output of tanh ($\in (-1, 1)$) provides natural clipping but can suffer from vanishing gradients for large hidden sizes.

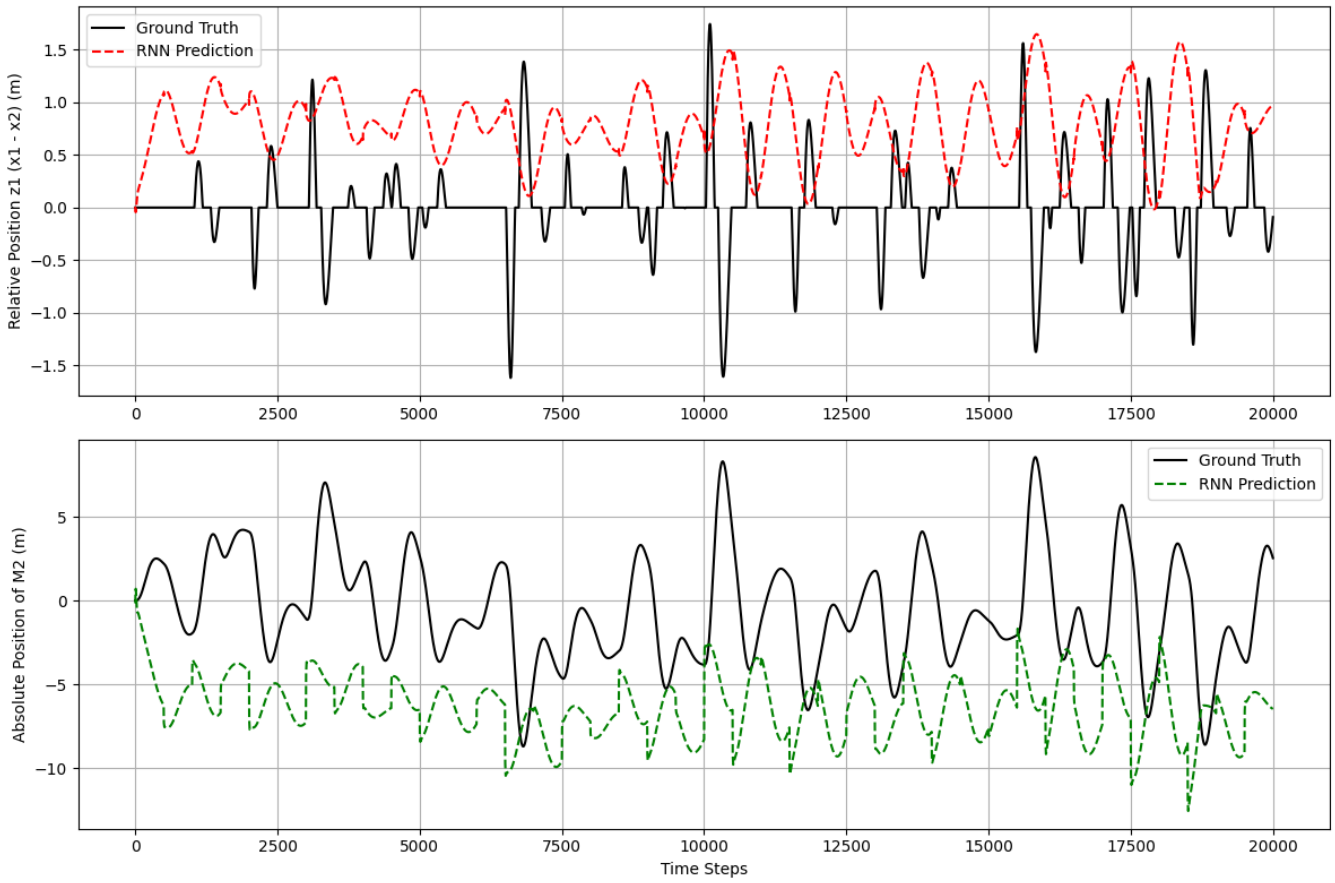

Figure 13: CL Deadzone/NLT RNN Evaluation (hidden 64, **z**-coords). The physics-inspired deadzone transform ($z_1 \leftarrow \text{deadzone}(z_1, 1.0)$) is applied to the relative position state before the hidden update, zeroing values in the deadzone region. This is a form of physics-informed feature engineering that exposes the steep linear spike in the spring structure more directly to the network.

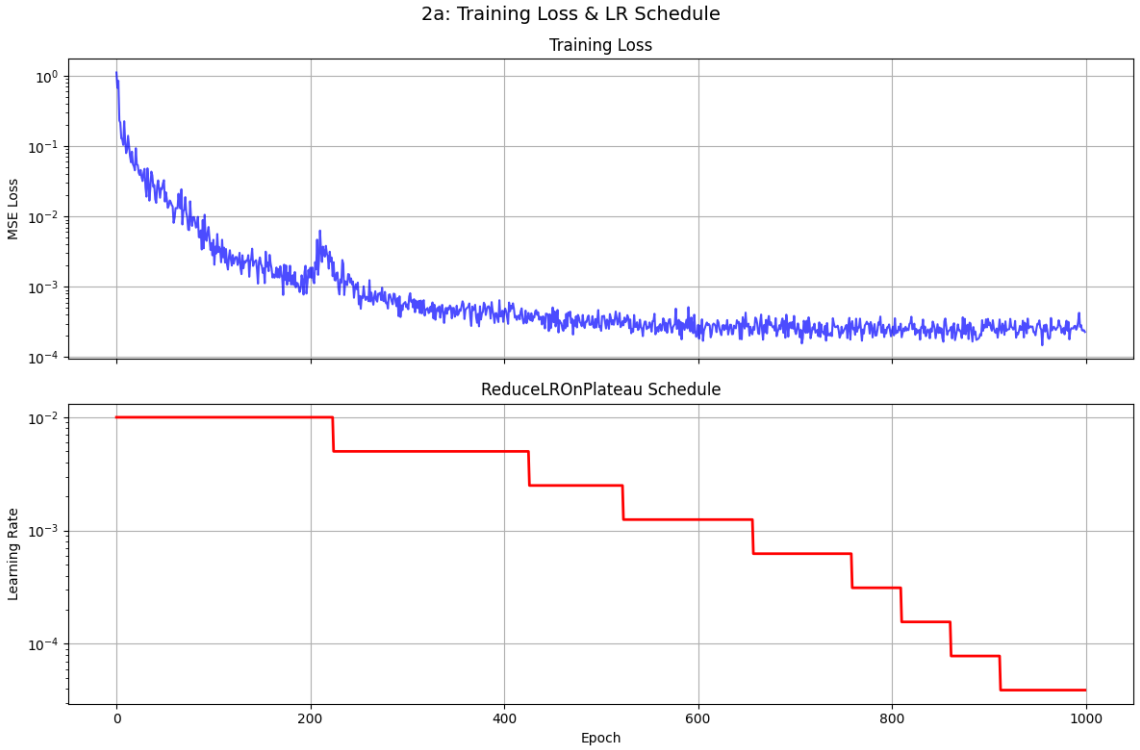

Figure 14: 2a Training Loss & LR Schedule (ReLU, h=64, scheduler). Top: training MSE loss on log scale. Bottom: `ReduceLROnPlateau` schedule. The LR halves each time the loss stagnates for 50 epochs, decreasing from 0.01 to 3.9×10^{-5} across seven reductions. Minimum loss: 1.47×10^{-4}.

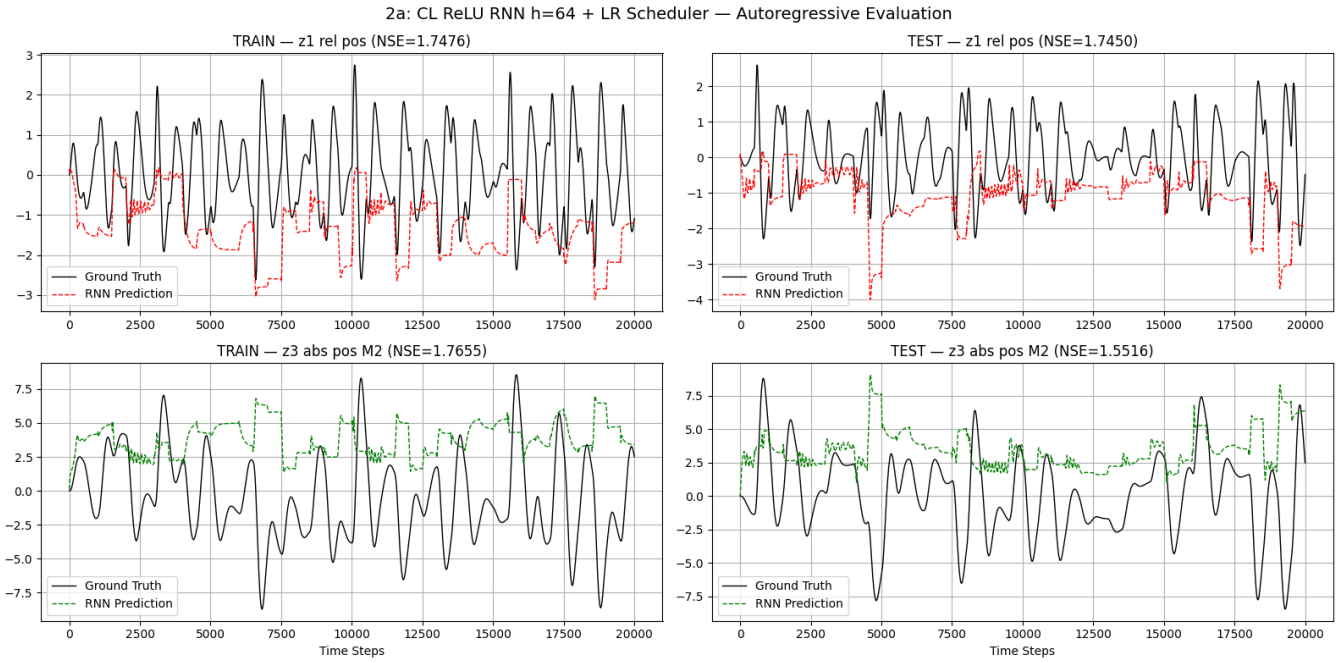

Figure 15: 2a Autoregressive Evaluation (divergence). Despite the low training loss, the closed-loop rollout diverges on both train (left) and test (right) data: NSE z_1=1.75, z_3=1.77 (train); z_1=1.75, z_3=1.55 (test). This reveals that an aggressive decrease in the LR over-optimizes the teacher-forced procedure, but the learned recurrence is not autonomously stable.

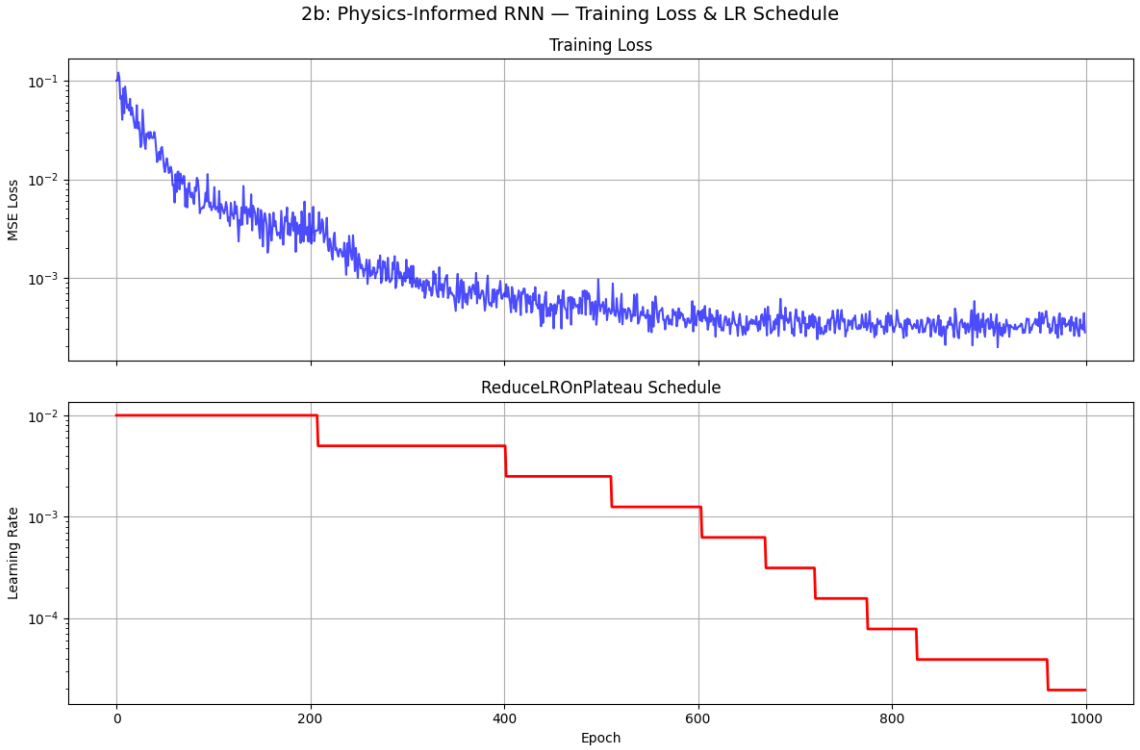

Figure 16: 2b Training Loss & LR Schedule (LS Residual, h=64). Same `ReduceLROnPlateau` as 2a but with 9-dim input $[\mathbf{z}_k; u_k; \hat{\mathbf{z}}_{k+1}^{\mathrm{LS}}]$. Minimum loss: 1.97×10^{-4}. The loss converges similarly, but the residual/error stabilizes over the autonomous rollout.

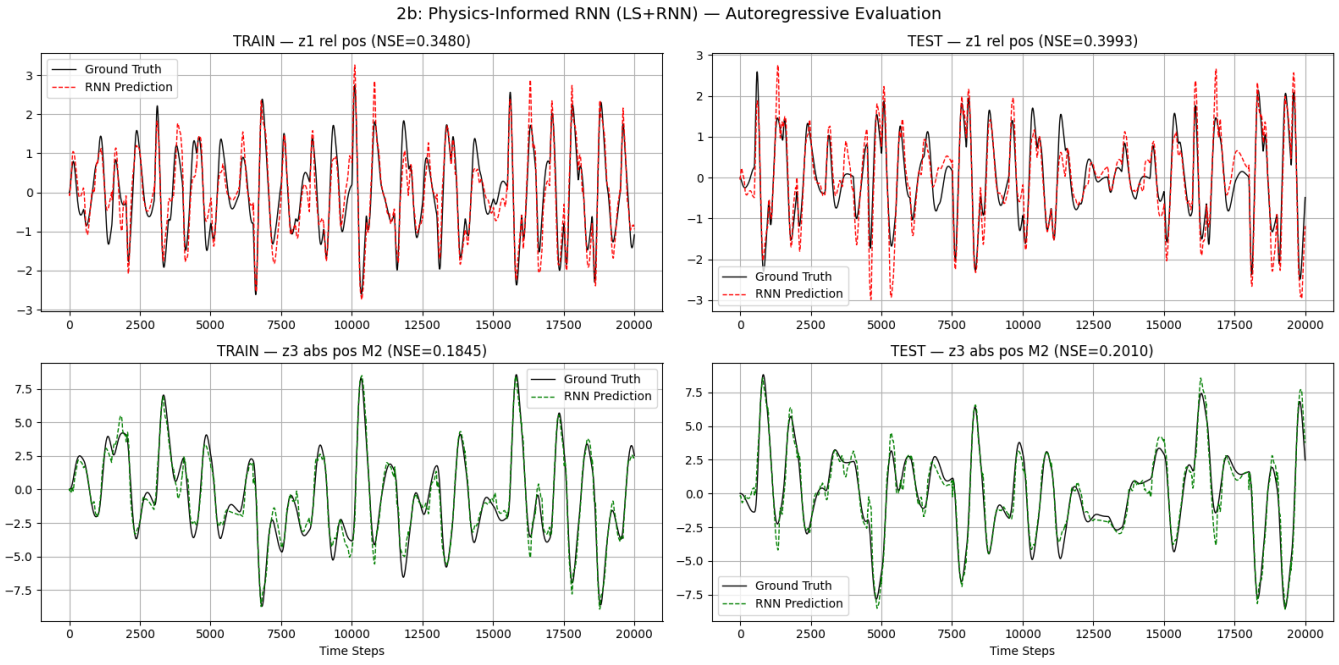

Figure 17: 2b Autoregressive Evaluation (best RNN result). The physics-informed residual model achieves stable closed-loop rollout: NSE z_1=0.35, z_3=0.18 (train); z_1=0.40, z_3=0.20 (test). The absolute position z_3 matches the LS test baseline (0.20). By providing the LS prediction as an extra input feature, the RNN learns only the nonlinear correction, greatly improving autonomous stability compared to 2a.